

instead on the underlying operand or descendent that is not also a negation. Otherwise, if naively approached, negation may have to populate a grounding list of all `False` or missing groundings from the underlying operand and store these as `True` under the closed-world assumption.

An inference context involves input operands and an output operation, where input operands are used in the upward inference pass to calculate a proof for the output, or where all but one input operand and the output are used in the downward inference pass to calculate a proof for the remaining input. If any participant in the inference context has a grounding that is not `Unknown`, then in real-valued logic it is possible in an inference context to derive a truth value that is also not `Unknown`. Each participant in the proof generation can thus add its groundings to the set of inference groundings. However, for complex reasoning problems that require long chains of reasoning it may be useful to also add groundings with `Unknown` states and propagate them in hopes that they fetch proofs from other neurons in the graph. These proofs can then be passed to other neurons in the graph to facilitate theorem proving. This introduces a trade-off between efficient computation, through avoiding propagating many constants, and handling complex reasoning problems, by allowing more constants propagation.

A given inference grounding is used as is for other participant operands with the same variable configuration as the originating operand. In case of disjoint variable(s) not present in the inference grounding, the overlapping variables are first searched for a match with all the disjoint variable values used in conjunction to create an expanded set of inference groundings. If no overlapping variables are present or no match is found, then the overlapping variables could be assigned according to the inference grounding, with the disjoint variable(s) covering its set of all observed combinations.

The set of relevant groundings from a real-valued inference context could become a significant expanded set, especially in the presence of disjoint variables. However, guided inference could be used to expand a minimal inference grounding set that only involves groundings relevant to a target proof, and manage the trade-off between computation and reasoning capacity. LNN can use a combination of goal-driven reasoning and data-driven reasoning³ to obtain a target proof. A backward-chaining-like algorithm is used here as a means of propagating groundings in search of known truth values that can then be used in a forward-chaining-like computation to infer the goal. If-then conditional rules typically require backward inference in the form of *modus tollens* to propagate groundings to the antecedent and *modus ponens* in the forward direction to help calculate the target proof at the consequent. This bidirectional chaining process continues until the target grounding at the consequent is not unknown or until inference does not produce proofs that are any tighter.

In summary, overall computation is characterized similar to the propositional case, with a few minor modifications:

1. Initialize neurons corresponding to predicates and formula roots with starting truth value bounds for given groundings. Usually, all formulae are initialized as `True` for all associated groundings. Ground atomic neurons with starting bounds represent input data.
2. For each formula, evaluate neurons in the forward direction in a pass from leaves to root, storing computed bounds at each node for all groundings, and propagating constants to other nodes in the same formula. Then, backtrack to each leaf using inverse computations to update subformula bounds based on stored bounds (and formula initial bounds), and propagate constants potentially fetched from other neurons in the graph.
3. Aggregate the tightest bounds computed at leaves for each proposition.
4. Return to step 2 until bounds for all groundings converge. Oscillation cannot occur because bounds tightening is monotonic.
5. Inspect computed bounds at specific predicates or formulae, i.e. those representing the predictions of the model.

³Forward-chaining and backward-chaining algorithms are special cases of data-driven and goal-driven reasoning, respectively, where logical formulae are represented using definite clauses.

A.6 Acceleration

As bounds tightening is monotonic, the order of evaluation does not change the final result. As a result, and in line with traditional theorem provers, computation may be subject to significant acceleration depending on the order that bounds are updated.

In order for such aggregate operations to be tractable, it is necessary to limit the number of key values that participate in computation, leaving other key value combinations in a sparse state, i.e. with default bounds. We achieve this by filtering predicates whenever possible to include only content pertaining to specific key values referenced in queries or involved in joins with other tables, prioritizing computation towards smaller such content. Because many truth values remain uncomputed in this model, the results of quantifiers and other reductions may not be tight, but they are nonetheless sound. In cases where predicates have known truth values for all key values (i.e. because they make the closed world assumption), we use different bounds for their sparse value and for the sparse values of connectives involving them, such that a connective's sparse value is its result for its inputs sparse values.

Even minimizing the number of key values participating in computation, it is necessary to guide neural evaluation towards rules that are more likely to produce useful results. A first opportunity to this effect is to shortcut computation if it fails to yield tighter bounds than were previously stored at a given neuron. In addition, we exploit the neural graph structure to prioritize evaluation in rules with shorter paths to the query and to visited rules with recently updated bounds.

Tensorization offers another route to accelerate computation, by formulating LNN in terms of weighted adjacency matrices and keeping truth values in multi-dimensional arrays indexed by node and grounding identifiers. The resulting truth value tensor can be sparse so that only entries of existing groundings are stored, and a further batch dimension corresponding to different universes and truth value initializations is also possible. A set of weighted adjacency matrices can represent the neural graph structure, with one adjacency matrix for each of the different operators, including conjunction, disjunction, and implication. The neuron weighting scheme can also extend to admit negative weights used such that corresponding inputs are logically negated.

B Examples of weighted real-valued logics

Weighted nonlinear logic is in fact only one family of possible logics implemented in LNN. Notably, the logic introduced in Section 5 for Theorem 2 is a generalization of weighted nonlinear logic in that its lower and upper bounds are computed by different functions. Another important family of logics satisfying the requirements for LNN are the *t-norm* logics, which we define presently.

Triangular norms, or *t-norms*, and their related *t-conorms* and *residua* are natural choices for LNN activation functions as they already behave correctly for classical inputs and have well known inference properties. Logics defined in terms of such function are denoted *t-norm logics*. Common examples of these include

	Gödel	Product	Łukasiewicz
T-norm	$\min\{x, y\}$	$x \cdot y$	$\max\{0, x + y - 1\}$
T-conorm	$\max\{x, y\}$	$x + y - x \cdot y$	$\min\{1, x + y\}$
Residuum	y if $x > y$, else 1	$\frac{y}{x}$ if $x > y$, else 1	$\min\{1, 1 - x + y\}$

Of these, only Łukasiewicz logic offers the familiar $(x \rightarrow y) = (\neg x \oplus y)$ identity, while only Gödel logic offers the $(x \otimes x) = (x \oplus x) = x$ identities.

B.1 Weighted Łukasiewicz logic

Weighted Łukasiewicz logic is exactly weighted nonlinear logic with $f(x) = \max\{0, \min\{1, x\}\}$. The binary and *n*-ary *weighted Łukasiewicz t-norms*, used for logical AND, are given

$$\beta(x_1^{\otimes w_1} \otimes x_2^{\otimes w_2}) = \max\{0, \min\{1, \beta - w_1(1 - x_1) + w_2(1 - x_2)\}\}, \quad (9)$$

$$\beta(\bigotimes_{i \in I} x_i^{\otimes w_i}) = \max\{0, \min\{1, \beta - \sum_{i \in I} w_i(1 - x_i)\}\} \quad (10)$$

for input set I , nonnegative bias term β , nonnegative weights w_i , and inputs x_i in the $[0, 1]$ range. By the De Morgan laws, the binary and n -ary *weighted Łukasiewicz t -conorms*, used for logical OR, are then

$$\beta(x_1^{\oplus w_1} \oplus x_2^{\oplus w_2}) = \max\{0, \min\{1, 1 - \beta + w_1x_1 + w_2x_2\}\} \quad (11)$$

$$\beta(\bigoplus_{i \in I} x_i^{\oplus w_i}) = \max\{0, \min\{1, 1 - \beta + \sum_{i \in I} w_i x_i\}\}. \quad (12)$$

In either case, the unweighted Łukasiewicz norms are obtained when all $w_i = \beta = 1$; if any of these parameters are omitted, their presumed value is 1. The exponent notation is chosen because, for integer weights k , this form of weighting is equivalent to repeating the associated term k times using the respective unweighted norm, e.g. $x^{\oplus 3} = (x \oplus x \oplus x)$. Bias term β is written as a leading exponent to permit inline ternary and higher arity-norms, for example $\beta(x_1^{\oplus w_1} \oplus x_2^{\oplus w_2} \oplus x_3^{\oplus w_3})$, which require only a single bias term to be fully parameterized.

The *weighted Łukasiewicz residuum*, used for logical implication, solves

$$\max\{z : y \geq \beta/w_y (x^{\otimes w_x/w_y} \otimes z^{\otimes 1/w_y})\} \quad (13)$$

and is given

$$\begin{aligned} \beta(x^{\otimes w_x} \rightarrow y^{\oplus w_y}) &= \max\{0, \min\{1, 1 - \beta + w_x(1 - x) + w_y y\}\} \\ &= \beta((1 - x)^{\oplus w_x} \oplus y^{\oplus w_y}). \end{aligned} \quad (14)$$

As for weighted nonlinear logic, note the use of $\otimes$ in the antecedent weight but $\oplus$ in the consequent weight, meant to indicate the antecedent has AND-like weighting (scaling its distance from 1) while the consequent has OR-like weighting (scaling its distance from 0). Similarly, this residuum is most disjunction-like when $\beta = 1$, most $(x \rightarrow y)$ -like when $\beta = w_y$, and most $(\neg y \rightarrow \neg x)$ -like when $\beta = w_x$; that is to say, $\beta = w_y$ yields exactly the residuum of $x^{\otimes w_x/w_y} \otimes z^{\otimes 1/w_y}$ (with no specified bias term of its own), while $\beta = w_x$ yields exactly the residuum of $(\neg y)^{\otimes w_y/w_x} \otimes z^{\otimes 1/w_x}$.

The Łukasiewicz norms are commutative if one permutes weights w_i along with inputs x_i , and are associative if bias term $\beta \leq \min\{1, w_i : i \in I\}$. Further, they return classical results, i.e. results in the set $\{0, 1\}$, for classical inputs under the condition that $1 \leq \beta \leq \min\{w_i : i \in I\}$. This clearly requires $\beta = 1$ to obtain both associative and classical behavior, though neither is a requirement for LNN. Indeed, constraining $\beta \leq w_i$ is problematic if we would like w_i to be able to go to 0, effectively removing i from input set I , whereupon the constraint should no longer apply. Section 6 presents a means of relaxing such constraints so as to facilitate learning.

B.2 Weighted Gödel logic

Gödel logic uses min and max for its t -norm and t -conorm, respectively. It is difficult in general to define weighted versions of these functions—for example, forms like $x^{\oplus 3} = (x \oplus x \oplus x)$ are not meaningful due to min and max’s idempotence—though several well-studied options exist, notably Fagin’s method [7]. This document, however, uses a technique borrowed from [11] to derive a weighted min from another t -norm, specifically the weighted Łukasiewicz t -norm. Hájek shows that, for any continuous t -norm, one may define *weak conjunction*

$$(x \wedge y) = (x \otimes (x \rightarrow y)) = \min\{x, y\}. \quad (15)$$

Using weighted Łukasiewicz logic and a specifically crafted configuration of weights⁴

$$(\beta_x (x^{\otimes w_x}) \wedge \beta_y (y^{\otimes w_y})) = \beta_x (x^{\otimes w_x} \otimes (x^{\otimes w_x/\kappa} \rightarrow y^{\oplus w_y/\kappa})^{\otimes \kappa}) \quad (16)$$

for $\kappa = \beta_x - \beta_y + w_y$, one obtains binary and n -ary *weighted Gödel t -norms*, defined

$$(\beta_1 (x_1^{\otimes w_1}) \wedge \beta_2 (x_2^{\otimes w_2})) = \max\{0, \min\{1, \beta_1 - w_1(1 - x_1), \beta_2 - w_2(1 - x_2)\}\}, \quad (17)$$

$$(\bigwedge_{i \in I} \beta_i (x_i^{\otimes w_i})) = \max\{0, \min\{1, \beta_i - w_i(1 - x_i) : i \in I\}\} \quad (18)$$

⁴This configuration of weights is the only one in which x may be swapped with y , w_x with w_y , and β_x with β_y without affecting the result; accordingly, it is the most reasonable adaptation of the above formulae to define weak conjunction for the weighted Łukasiewicz t -norm.